

CS603 - Software Architecture and Design

Midterm Study Notes

Topics 01 to 61 | Lectures 1 to 18 | Spring 2026

WEEK 1 — Software Design Concepts

| Topic 01 — Design — What and Why

Design means making a plan before building something. Just like an architect draws a blueprint before constructing a building, a software engineer designs the system before writing code.

Example: Frank Lloyd Wright: 'You can use an eraser on the drafting table or a sledgehammer on the construction site.' — Fix mistakes during design, not after coding.

| Topic 02 — Design — Objectives

Software design takes System Specification (what the client wants) and converts it into a blueprint for implementation (how to build it).

Goals of good software design:

- Meet customer needs and desires
- Be easy to implement
- Be efficient in performance
- Be easily extendable for future requirements

| Topic 03 — Software Design — Complexity

Poorly designed programs are difficult to understand and modify. As you keep adding features to a badly designed program, the cost grows exponentially.

A well-designed program stays manageable even as it grows. Good design = lower long-term cost.

| Topic 04 — Software Design — Complexity and Size

The bigger the program, the more severe the consequences of poor design. Size matters — a large badly-designed system becomes impossible to maintain.

| Topic 05 — Types of Complexity

Essential Complexity: Complexity that naturally comes from the problem itself. Cannot be avoided.

Accidental/Incidental Complexity: Complexity added due to poor design decisions. Can and should be avoided.

Total Complexity = Essential Complexity + Accidental Complexity

Good design reduces accidental complexity without removing essential complexity.

Example: A banking system is inherently complex (essential). Writing messy, tangled code makes it worse (accidental).

Topic 06 — Why is Software Design Hard?

Design is hard because you are creating an abstraction of a solution that does not exist yet. You are imagining and planning something before it is built.

You cannot fully see the problems until you try to solve them.

Topic 07 — Design — A Science or an Art?

Software design is both a science and an art. It follows scientific principles and methods, but also requires creativity, intuition, and experience.

Two designers given the same problem will likely produce different designs — and both can be correct.

Topic 08 — Design — A Wicked Problem

A wicked problem is one that can only be clearly defined by solving it. Design is a wicked problem — you only fully understand the problem after working through the solution.

Even expert designers cannot always explain exactly how they arrived at their design. It comes through experience and iteration.

Topic 09 — Making Design More Systematic

To make the design process more predictable and repeatable, we use:

- Methods — structured processes to follow
- Techniques — specific tools and approaches
- Patterns — reusable solutions to common problems
- Principles and Heuristics — rules of thumb that guide good decisions

Topic 10 — Modularity, Hierarchy, Information Hiding, Abstraction

Good design does not remove essential complexity but manages it so a programmer deals with only a small piece at a time.

Modularity: Break the solution into smaller, manageable pieces (divide and conquer).

Hierarchical Organization: Larger components are made of smaller ones (like folders containing files).

Information Hiding: Hide internal details behind simple interfaces. You use a TV remote without knowing its electronics.

Abstraction: Suppress unnecessary details. Focus only on what matters at that level.

Topic 11 — Characteristics of Software Design

Non-deterministic: No two designers produce the same design. Unlike math, there is no single correct answer.

Heuristic: Design uses rules of thumb, not guaranteed formulas.

Emergent: The final design evolves through experience, feedback, and iteration. It is not created in one step.

Topic 12 — Benefits of Good Design

- Reduced complexity — easier to understand and modify
- Rapid development and easier maintenance
- Code reuse across projects
- Better software quality — easier to test
- Better security — complex code has more vulnerabilities

Topic 13 — A Generic Design Process

The 6-step design process:

- Step 1: Understand the problem (read requirements)
- Step 2: Build a black-box model using use cases
- Step 3: Look for existing patterns and solutions
- Step 4: Discover missing requirements, build prototypes
- Step 5: Document and review the design
- Step 6: Iterate and refactor until all requirements are met

Topic 14 — Inputs to the Design Process

User Requirements: What the client wants, including constraints on design and implementation.

Domain Knowledge: Understanding the field (e.g., healthcare terms for a hospital application).

Implementation Knowledge: Knowing the capabilities and limitations of the target technology or environment.

Topic 15 — Desirable Internal Design Characteristics

Minimal Complexity: Keep it simple. Do not over-engineer.

Extensibility: Design for today but keep the future in mind.

Loose Coupling: Minimize dependencies between modules.

Reusability: Build things that can be reused in other contexts.

Portability: Works or can easily work in other environments.

Note: Extensibility and minimal complexity can conflict. Engineering is about balancing such trade-offs.

WEEK 2 — Coupling, Cohesion and Design Strategies

Topic 16 — What is Good Design?

Good design means Maintainable Design. Specifically:

- Cost of making changes is minimal
- Easy to add or modify functionality
- Design is understandable by other developers
- Changes have local effect — do not break unrelated parts

This is achieved through High Cohesion and Low Coupling.

Topic 17 — Coupling

Coupling measures how dependent two modules are on each other. If changing one module forces changes in another, they are tightly coupled.

High coupling is bad: difficult to test, reuse, or modify modules independently.

Goal: Low coupling — make modules as independent as possible.

How to reduce coupling:

- Eliminate unnecessary relationships
- Reduce the number of necessary relationships
- Simplify the nature of necessary relationships

Topic 18 — Coupling — Example (Web App)

Example comparing two approaches for changing webpage style:

Option A (Bad): JavaScript directly modifies style attributes (color, font). Tight coupling between JS and CSS.

Option B (Good): JavaScript only changes the CSS class name. CSS handles all styling. Loose coupling.

Example: If you change the visual style in Option B, you only edit CSS. JS does not need to change at all.

Topic 19 — Cohesion

Cohesion measures how well the elements inside a single module belong together and work toward the same goal.

High Cohesion: All methods and data in a class serve one clear purpose. Good design.

Low Cohesion: Class does many unrelated things. Difficult to understand, test, and reuse.

Example: A class called UserManager that handles login, email sending, AND payment has low cohesion. Split into UserAuth, EmailService, and PaymentService.

Topic 20 — Relationship Between Coupling and Cohesion

They are inversely related: when coupling is high, cohesion tends to be low, and vice versa.

Best design = High Cohesion within a module + Low Coupling between modules.

Benefits: easier to read, modify, test, develop, and reuse.

Topic 21 — Software Design Strategies — Structured vs Object-Oriented

Structured Design: System designed as a set of functions. System state is shared/centralized. Action-oriented approach.

Object-Oriented Design (OOD): System is a collection of objects. Each object manages its own state and intelligence. Objects communicate through messages.

OOD is better for maintainability as it naturally supports low coupling and high cohesion.

Topic 22 — OO Basic Concepts — The Object

An Object is a real or abstract entity with a well-defined role in the problem domain.

Every object has three things:

State: Current values of its properties (e.g., a bank account's current balance).

Behavior: How it acts and reacts (e.g., deposit, withdraw methods).

Identity: What makes it unique from all other objects.

Topic 23 — OO Basic Concepts — The Class

A Class is a template or blueprint for creating objects. All objects of a class share the same structure and behavior.

Interface (public view): What the class can do for you — the operations it offers to outside callers.

Implementation (private view): How it actually does it — the internal data structure and algorithm.

Example: 'Car' is a class. Your specific car with license plate ABC-123 is an object (instance) of that class.

WEEK 3 — OOP Relationships and UML

Topic 24 — OO Relationships — Association

Association is a general relationship between two classes, meaning objects of one class are connected to objects of another.

It does NOT mean all objects are always linked — just that a persistent connection can exist.

Has attributes: name, direction (one-way or two-way), roles, and multiplicity (how many of each side).

Example: Student is associated with Course. One student can enroll in many courses.

Topic 25 — OO Relationships — Aggregation and Composition

Aggregation: A whole-part relationship shown with a hollow diamond in UML. The part can exist independently of the whole. Most experts consider it a modeling placebo.

Composition: A strong whole-part relationship shown with a filled diamond in UML. The part CANNOT exist without the whole. If the whole is destroyed, so are the parts.

Example: Composition: A House has Rooms — Rooms cannot exist without the House.
Aggregation: A University has Students — Students exist independently.

Topic 26 — OO Relationships — Inheritance

Inheritance allows one class (subclass) to inherit the structure and behavior of another class (superclass).

This creates a generalization/specialization hierarchy.

The ISA Rule: a subclass IS-A superclass. Example: Dog IS-A Animal.

Subclasses can add new features or override existing behavior from the superclass.

Example: Animal (superclass) → Dog, Cat (subclasses). Dog inherits all Animal properties but adds bark() and fetch().

Topic 27 — Abstract Classes and Interfaces

Abstract Class: A partially implemented class. You know the structure but not all details. You CANNOT create an object directly from it — only from its concrete subclasses.

Interface: A collection of abstract method signatures only. No state, no implementation. Any class implementing it must provide all method implementations.

Key difference: Abstract classes can have some implementation and state; interfaces have none.

Topic 28 — OOP Defined (Grady Booch)

OOP organizes programs as cooperative collections of objects, each representing an instance of a class, where classes are united via inheritance relationships.

Inheritance: Relationship among classes.

Association: Relationship among instances (objects).

Topic 29 — Association — Detailed

Association is the glue that holds a system together. Without associations, you have only isolated, unconnected classes.

Types of Association:

- Uses relationship — one class uses another
- Simple association — general link
- Aggregation — whole-part (weak)
- Composition — whole-part (strong)

Topic 30 — Aggregation and Composition — Detailed

Aggregation: A whole-part relationship where the part can survive independently. Martin Fowler (UML Distilled) says: 'Aggregation is strictly meaningless — I recommend you ignore it.'

Composition: Strong ownership. Parts share the lifetime of the whole. If the composite is destroyed, so are all its parts. Each part belongs to only ONE composite at a time.

Topic 31 — Object Lifetime — Inheritance

When a subclass object B is created, a corresponding superclass object A is automatically created inside it.

The A object is exclusively held by B. It is created with B and destroyed with B. Cardinality is always 1:1.

Topic 32 — Object Lifetime — Composition

When the composite B is created, its part objects A are also created inside it. Parts are exclusively held and cannot be accessed from outside B.

Parts are created and destroyed with the composite. Cardinality: 1:1 or 1:many.

Topic 33 — Object Lifetime — Association

In association, linked objects have independent lifetimes. Object B links to object A, but they are created and destroyed separately.

The relationship is NOT exclusive — the same A object can be linked to multiple B objects simultaneously.

Cardinality: 1:1, 1:many, or many:many.

WEEK 4 — Object to Relational Mapping and Polymorphism

Topic 34 — Object to Relational Mapping — Basics

ORM (Object to Relational Mapping) defines how to store OO objects in a relational database. It covers:

- Mapping attributes (object fields to table columns)
- Mapping Inheritance hierarchies
- Mapping Associations between objects
- Mapping Composition relationships

Topic 35 — Mapping Inheritance — Three Strategies

Filtered Mapping: All classes in one table with extra columns for each subclass. Simple but wastes space with many NULL values.

Horizontal Mapping: Each concrete class gets its own table including all inherited attributes. Duplicates data from parent.

Vertical Mapping: Each class (including abstract) gets its own table. Child tables use parent OID as a foreign key. Most normalized approach.

Topic 36 — Mapping Inheritance — Vertical Mapping Detail

Each class maps to its own table. The child table contains a Foreign Key (FK) pointing to the parent table's primary key (OID).

To reconstruct an object, you JOIN the parent and child tables using the FK.

Example: TableA(oid_a, u, v) and TableB(oid_b, oid_a_FK, x, y). Each row in B points to its corresponding row in A.

Topic 37 — Mapping Composition

Each class maps to its own table. The PART table contains a FK pointing to the WHOLE/COMPOSITE table.

Note the difference from inheritance: in inheritance the child has FK to parent; in composition the part has FK to the composite.

Topic 38 — Mapping Association

Each class maps to its own table. The used class stores the FK of the user class.

For many-to-many relationships, a separate junction table is created to hold both foreign keys.

Topic 39 — Mapping Cardinality of Association

One-to-One: FK is placed in one of the two tables.

One-to-Many: FK is placed in the 'many' side table. Example: Invoice table holds CustomerOID FK.

Many-to-Many: A separate join table is created with both FKs forming a composite primary key.

Topic 40 — ORM Summary Table

Quick comparison of the three object relationships:

- Inheritance (ISA) — 1:1 exclusive lifetime, child has FK to parent
- Composition (whole-part) — exclusive, parts die with whole, part has FK to composite
- Association (using link) — non-exclusive, independent lifetimes, FK or join table

Topic 41 — OOP — Inheritance and Polymorphism

Polymorphism is the ability to treat objects of different subclasses through a common base class reference.

Using Polygon* for both Rectangle and Triangle objects, calling printarea() automatically calls the correct version at runtime based on the actual object type.

Example: Benefit: Store many different object types in one array of Polygon* and process all of them with one loop.

Topic 42 — OOP — Object Creation and Factory Method

Object creation is NOT polymorphic by default — you must explicitly write 'new Rectangle' or 'new Triangle'.

Factory Method: A function that takes a type parameter and creates the correct object. This centralizes object creation.

Example: createPolygon('R', w, h) returns a Rectangle. createPolygon('T', w, h) returns a Triangle. The caller does not need to know the details.

Topic 43 — OOP — The Magic Behind Polymorphism (vtable/vptr)

The compiler maintains two things for polymorphism:

vtable: A table of function pointers maintained per CLASS. Each class has its own vtable listing virtual method addresses.

vptr: A pointer to the vtable, maintained per OBJECT. When a virtual function is called, the program follows vptr to vtable to find the correct function.

This mechanism is how the correct method version is called even when using a base class pointer.

WEEK 5 — Implementing OOP + SOLID Principles (S, O)

Topic 44 — OOP — Implementing Composition

Composition is implemented by including objects of the component class as private member variables of the composite class.

Component objects are created in the composite's constructor using an initializer list. They live inside the composite and die with it.

Example: Event class contains Time and Date objects as private members. They are initialized in the Event constructor and destroyed when the Event is destroyed.

Topic 45 — OOP — Inheritance vs Composition

Inheritance (IS-A): Derived class IS-A base class. Allows dynamic binding and polymorphism. Can override virtual methods.

Composition (HAS-A): Aggregate class HAS-A component. Cannot override virtual methods of the component.

Main advantage of inheritance: enables polymorphism through dynamic binding. Composition cannot do this.

Topic 46 — OOP — Implementing Unidirectional Associations

In a unidirectional association, one class knows about the other but not vice versa.

Implemented by storing a pointer to the associated object as a private member. Objects are created separately and then linked.

Example: Course has a pointer to a Lecturer (L). addLecturer(teacher) sets L = teacher. removeLecturer() sets L = NULL. Lecturer does not know about Course.

Topic 47 — OOP — Implementing Bidirectional Associations

In a bidirectional association, both classes know about each other. Both hold a pointer to the other.

Problem: updating both ends manually is error-prone. Solution: use the 'this' pointer to automatically update the other side.

Problem: infinite recursion. Solution: check if the pointer is already set (NULL check) before setting it to prevent infinite recursive calls.

Topic 48 — SOLID Design Principles — Introduction

SOLID is a set of 5 design principles that make software more Understandable, Flexible, and Maintainable. Coined by Robert C. Martin (Uncle Bob).

S: Single Responsibility Principle (SRP) — Increased Cohesion

O: Open-Closed Principle (OCP) — Flexibility and Maintainability

L: Liskov Substitution Principle (LSP) — Proper Inheritance

I: Interface Segregation Principle (ISP) — Reduce Coupling

D: Dependency Inversion Principle (DIP) — Reusable Frameworks

Topic 49 — SOLID — Single Responsibility Principle (SRP)

A class should have only ONE reason to change.

If a class does multiple things, changing one thing might break another. More responsibilities = more reasons to change = more bugs.

Find and separate responsibilities — this is much of what software design is really about.

| **Topic 50 — SOLID — SRP Example**

Bad: A Post class that creates posts AND handles error logging — two responsibilities in one class.

Good: Split into two classes:

- Post class — only responsible for creating posts
- ErrorLogger class — only responsible for logging errors

Post uses ErrorLogger through composition. Each class has exactly one clear job.

WEEK 6 — SOLID Principles (O, L, I, D) + Law of Demeter

Topic 51 — SOLID — Open-Closed Principle (OCP)

Software should be Open for Extension but Closed for Modification.

You should be able to add new behavior WITHOUT changing existing working code. Use inheritance, composition, or association to extend behavior.

If adding a small new requirement forces massive changes to existing code, the architecture has failed.

Topic 52 — SOLID — OCP Example

Bad: Post class has an if/else block checking for '#' to handle tags. Adding '@' mentions requires modifying the existing class — violates OCP.

Good: Create a base Post class and separate subclasses (TagPost, MentionPost) that extend it. Adding a new post type means creating a new subclass only — no changes to existing code.

Topic 53 — SOLID — Liskov Substitution Principle (LSP)

Subclasses must be substitutable for their base classes without breaking the program.

If a function works with a base class, it must work correctly with any subclass too — without the caller knowing or caring.

Example: Violation: DogNoWag inherits from Dog but wagTail() does nothing. Code that expects a real Dog's behavior breaks when given a DogNoWag.

Topic 54 — SOLID — LSP Example (Square/Rectangle)

Classic violation: Square inherits from Rectangle. Setting the width of a Square also changes the height to keep it square. A function expecting a Rectangle gets wrong area calculations when passed a Square.

Though a Square IS-A Rectangle mathematically, behaviorally it is NOT in this design.

LSP teaches: ISA in OOD is about behavior, not just mathematical relationships.

Topic 55 — SOLID — LSP and the True Meaning of ISA

A model cannot be validated in isolation — only in the context of how clients (callers) use it.

Square and Rectangle seemed valid when viewed alone, but broke when used by a programmer making reasonable base-class assumptions.

Key lesson: LSP is about extrinsic public behavior that clients depend upon, not just internal private behavior.

Topic 56 — SOLID — Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

Do not add new methods to an existing interface. Instead, create a new interface. Classes implement only the interfaces they actually need.

Fat (bloated) interfaces cause inadvertent coupling between unrelated clients. Break them into smaller, cohesive interfaces.

Topic 57 — SOLID — ISP Example

Bad: IPost interface starts with CreatePost(). Then ReadPost() is added to it. Old clients that only needed CreatePost() are now forced to know about ReadPost() — violates ISP.

Good: Create two separate interfaces:

- IPostCreate — contains only CreatePost()
- IPostRead — contains only ReadPost()

Classes implement only the interfaces they need. If both are needed, the class implements both.

Topic 58 — SOLID — Dependency Inversion Principle (DIP)

Rule 1: High-level modules should NOT depend on low-level modules. Both should depend on abstractions.

Rule 2: Abstractions should not depend on details. Details should depend on abstractions.

In practice: depend on interfaces or abstract classes, not on concrete implementations.

Usually implemented via Dependency Injection — inject dependencies through the constructor instead of creating them inside the class.

Topic 59 — SOLID — DIP Example

Bad: Post class creates its own ErrorLogger object inside the class — tightly coupled. To change the logger type, you must modify Post.

Good: Pass the Logger as a constructor parameter (dependency injection). Post only knows about an abstract Logger — you can swap different logger implementations without changing Post at all.

Topic 60 — Law of Demeter (LoD)

The Law of Demeter says: 'Do not talk to strangers.' Each module should only communicate with its immediate friends.

A method M in class C should only interact with:

- C itself (the self/this object)
- M's own parameters
- Objects created inside M
- C's own instance variables

This principle reduces hidden dependencies and coupling, making the system more robust to changes.

Side effect: you may need to write many small wrapper methods — but the result is much more maintainable code.

Topic 61 — Law of Demeter — Example

Bad: Register calls sale.getPayment().getTenderedAmount() — Register is accessing Payment through Sale. Payment is a 'stranger' to Register.

This creates a hidden dependency — Register secretly depends on Payment even though it is not visible in the class diagram. This increases coupling.

Good: Add a getPaymentAmount() method in Sale, which internally calls payment.getTenderedAmount() and returns the result. Register only talks to Sale — its direct friend.

Example: Result: Register does not know how the payment amount was obtained. Information hiding is preserved and coupling is reduced.

End of CS603 Midterm Notes — Good Luck!